

Dashboard Alertas (Parte del sistema C5_ALERT_SYSTEM22)

El **Dashboard de Alertas** es la interfaz gráfica en tiempo real de la plataforma **Vanguardia C5**. Desarrollado con React, Vite y Tailwind CSS, se conecta directamente a las APIs del backend para recibir de forma instantánea los reportes de incidentes, organizándolos visualmente en tarjetas interactivas clasificadas por colores según su nivel de prioridad (*Crítico, Alto, Medio*), además de integrar un panel de administración para calibrar un motor de Inteligencia Artificial (NLP) encargado de procesar y filtrar de forma automatizada los flujos de emergencias entrantes.

Su propósito principal dentro del ecosistema del C5 es **reducir al mínimo los tiempos de respuesta institucional ante crisis y optimizar la asignación de recursos en campo**. La aplicación opera como una consola táctica unificada que elimina la dispersión de datos crudos del sistema, permitiendo a los agentes operativos examinar rápidamente evidencias multimedia complejas (audios o fotografías de cámaras de monitoreo) y despachar de inmediato el tipo de apoyo requerido (Patrullas, Ambulancias o Bomberos) hacia el cuadrante o sector geográfico exacto de la incidencia.

El diseño del flujo y los accesos de la plataforma están orientados exclusivamente a cumplir las necesidades de tres perfiles de usuarios finales en el centro de comando: los **Operadores de Monitoreo (Monitoristas)**, que vigilan la pantalla de forma permanente para validar la veracidad de las alarmas entrantes; los **Despachadores de Emergencia**, encargados de gestionar el ciclo de vida del incidente actualizando sus estatus (*Pendiente, En Proceso, Atendida*) y coordinar el apoyo táctico; y los **Administradores de Sistemas (DevOps)**, quienes se ocupan del soporte técnico interno, la modificación de parámetros del modelo de IA y el despliegue estable mediante contenedores Docker.

Stack Tecnológico

El entorno de desarrollo y compilación del Frontend está estructurado bajo un ecosistema moderno de herramientas interconectadas que garantizan un rendimiento óptimo, alta velocidad de respuesta visual y la homogeneidad en la estructura del código fuente.

Librería Principal: React

Constituye el núcleo del desarrollo de la interfaz de usuario, permitiendo un modelo basado en componentes reutilizables y un manejo dinámico y reactivo del estado global y local. Su implementación queda completamente evidenciada en la arquitectura a través de los archivos raíz `main.jsx` (punto de entrada e instanciación del DOM virtual) y `App.jsx` (componente contenedor principal de la lógica del dashboard).

Punto de Entrada (`src/main.jsx`): Es el encargado de arrancar el frontend, conectarse con el DOM del archivo `index.html` e inicializar el árbol de componentes de React.

```
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import './index.css'
4  import App from './App.jsx'
5
6  createRoot(document.getElementById('root')).render(
7    <StrictMode>
8      <App />
9    </StrictMode>,
10 )
```

Componente Raíz (`src/App.jsx`): Constituye el contenedor maestro de la aplicación, controlando las vistas dinámicas de las tarjetas de alertas, los contadores de analítica superior y el flujo visual del panel de control de Vanguardia C5.

Herramienta de Construcción: Vite

Como motor de automatización de desarrollo y empaquetado (Build Tool) se utiliza Vite. Reemplaza a las herramientas tradicionales al ofrecer un servidor de desarrollo

ultra veloz basado en ESM nativo y un empaquetado final optimizado para producción mediante *Rollup*. Su comportamiento y resolución de rutas están definidos en el archivo maestro de configuración de la raíz:

Archivo de Configuración (vite.config.js):

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vitejs.dev/config/
5 export default defineConfig({
6   plugins: [react()],
7   server: {
8     host: '0.0.0.0', // Esto es el equivalente a "0.0.0.0", crucial para Docker
9     port: 5173,
10    watch: {
11      usePolling: true // Ayuda a que los cambios en el código se reflejen al instante en Wind
12    }
13  }
14 })
```

Estilos y Maquetación: Tailwind CSS

El diseño visual interactivo, adaptado a un entorno de alta concentración visual en modo oscuro, se implementó mediante Tailwind CSS. Este framework permite aplicar estilos directamente sobre los componentes mediante clases de utilidad, optimizando el tamaño del paquete final al eliminar las reglas CSS que no se usan en producción. Su configuración y el procesamiento del diseño se gestionan mediante dos archivos clave:

Archivo de Directivas (tailwind.config.js): Controla los archivos que el framework debe inspeccionar para aplicar los estilos.

```
1 /** @type {import('tailwindcss').Config} */
2 export default {
3   content: [
4     './index.html',
5     './src/**/*..{js,ts,jsx,tsx}',
6   ],
7   theme: {
8     extend: {},
9   },
10  plugins: [],
11 }
```

Archivo del Procesador (postcss.config.js): Automatiza la inyección de prefijos de compatibilidad para navegadores modernos a través de Autoprefixer.

```
1  export default {
2    plugins: {
3      tailwindcss: {},
4      autoprefixer: {},
5    },
6  }
```

Calidad de Código: ESLint

Para garantizar la consistencia en la sintaxis, evitar la acumulación de código muerto o propenso a errores y mantener un estándar limpio dentro de las contribuciones del equipo de desarrollo, se configuró la herramienta de análisis estático ESLint. Las reglas generales, entornos de ejecución soportados y plugins específicos para componentes reactivos se declaran en la raíz del proyecto:

Archivo de Validación (eslint.config.js):

```
1  import js from '@eslint/js'
2  import globals from 'globals'
3  import reactHooks from 'eslint-plugin-react-hooks'
4  import reactRefresh from 'eslint-plugin-react-refresh'
5  import { defineConfig, globalIgnores } from 'eslint/config'
6
7  export default defineConfig([
8    globalIgnores(['dist']),
9    {
10     files: ['**/*.js', '**/*.jsx'],
11     extends: [
12       js.configs.recommended,
13       reactHooks.configs.flat.recommended,
14       reactRefresh.configs.vite,
15     ],
16     languageOptions: {
17       globals: globals.browser,
18       parserOptions: { ecmaFeatures: { jsx: true } },
19     },
20   },
21 ])
22
```

Contenedores e Infraestructura: Docker

La portabilidad, consistencia y eficiencia en producción del frontend se resuelven mediante el uso de una arquitectura de construcción multi-etapa (Multi-stage build) en Docker. Este enfoque optimiza drásticamente el tamaño de la imagen final al separar el entorno de desarrollo del servidor web de producción:

- Etapa 1: Construcción (Builder): Utiliza una imagen ligera de node:22-alpine para instalar las dependencias con npm install y ejecutar la compilación del dashboard con npm run build, generando los archivos estáticos listos para producción dentro del directorio /app/dist.
- Etapa 2: Servidor de Producción (Nginx): Utiliza un servidor web Nginx de alta disponibilidad sobre Alpine. Descarta por completo el entorno pesado de Node.js y se limita a copiar exclusivamente los archivos estáticos compilados de la etapa anterior hacia la ruta pública de Nginx (/usr/share/nginx/html), exponiendo la aplicación de manera segura e inmutable a través del puerto estándar 80.

```
1  # --- Etapa 1: Construcción (Build) ---
2  FROM node:22-alpine AS builder
3  WORKDIR /app
4  COPY package*.json ./
5  RUN npm install
6  COPY . .
7  # Esto genera los archivos estáticos en la carpeta /app/dist
8  RUN npm run build
9
10 # --- Etapa 2: Servidor de Producción (Nginx) ---
11 FROM nginx:alpine
12 # Copiamos los archivos compilados de la etapa anterior al directorio público de Nginx
13 COPY --from=builder /app/dist /usr/share/nginx/html
14
15 # Exponemos el puerto 80
16 EXPOSE 80
17 CMD ["nginx", "-g", "daemon off;"]
```

Requisitos previos

Antes de proceder con la clonación, instalación y despliegue del Dashboard de Alertas, es imperativo que el entorno local del desarrollador cuente con las siguientes herramientas de software debidamente instaladas y configuradas en el sistema.

Dado que el Dockerfile de producción utiliza como base la versión de Node 22, las especificaciones recomendadas para garantizar la compatibilidad absoluta del entorno son:

- Node.js (Versión Recomendada: v22.x LTS o superior): Es el entorno de ejecución para JavaScript en el servidor indispensable para ejecutar el servidor de desarrollo local de Vite y orquestar las tareas de compilación. No se recomiendan versiones inferiores a la v18 debido a incompatibilidades con las reglas sintácticas del archivo de configuración `eslint.config.js`.
- Gestor de Paquetes (npm v10.x o superior): Incluido por defecto con la instalación de Node.js. Es la herramienta encargada de resolver, descargar y administrar el árbol de dependencias del proyecto basándose estrictamente en los archivos de bloqueo `package.json` y `package-lock.json`.
- Docker Desktop / Docker Engine (Versión v25.x o superior): Necesario para construir la imagen multi-etapa y levantar el contenedor local que emula el servidor de producción con Nginx. Debe incluir soporte para Docker Compose en caso de integraciones locales concurrentes con las APIs del backend.

Estructura de Directorios

Para guiar a los nuevos desarrolladores en la navegación del proyecto y mantener un orden modular a medida que la plataforma escale, se detalla a continuación la arquitectura de carpetas y archivos clave que estructuran la raíz del Frontend:

Directorios Raíz

- **public/ (Recursos Estáticos Puros)**
 - **Propósito Técnico:** Aloja archivos que se sirven de forma directa e idéntica en la raíz de la URL del servidor web (Nginx) sin ser modificados.
 - **Comportamiento:** Estos recursos **no pasan** por el proceso de empaquetado, minificación o transformación de Vite. Es el lugar exclusivo para el archivo favicon.ico, el manifiesto web o los iconos de notificaciones nativas que el navegador necesita leer de manera estática e inmediata.
- **src/ (Núcleo del Código Fuente)**
 - **Propósito Técnico:** Es el contenedor absoluto de toda la lógica de negocio, los estados reactivos, las hojas de estilo de diseño y los componentes que dan vida a la Single Page Application (SPA).
 - **Comportamiento:** Todo el contenido de este directorio es monitoreado por el servidor de desarrollo para aplicar *Hot Module Replacement* (HMR) y es compilado y transformado en código JavaScript optimizado (ES6+) durante el proceso de *build*.

Subdirectorios de la Aplicación (src/)

- **src/assets/ (Recursos Multimedia Compilados)**
 - **Propósito Técnico:** Almacena los elementos visuales dinámicos que utiliza la interfaz, como el logotipo oficial de "Vanguardia C5" o iconos vectoriales.
 - **Comportamiento:** A diferencia de public/, estos archivos **sí son procesados por Vite**. Al compilarse, Vite les asigna un hash único en el nombre (ej. logo-X7y2z.png) para romper la caché del navegador al actualizar el sistema y optimiza su tamaño.
- **src/components/ (Arquitectura Modular de la Interfaz)**

- **Propósito Técnico:** Centraliza los bloques atómicos e interactivos que componen la pantalla táctil de monitoreo. Está integrada estrictamente por tres componentes tácticos de alto impacto:
 - **AlertCard.jsx (Tarjeta de Incidente):** Componente reactivo encargado de renderizar y dar formato a cada alerta que ingresa al panel. Evalúa el payload del incidente y altera dinámicamente sus bordes y etiquetas con colores tácticos basados en Tailwind CSS según su severidad: **Rojo** para prioridad *Crítico*, **Naranja** para *Alto* y **Amarillo** para *Medio*. Incluye los accesos directos de despacho rápido y los botones de acción para auditar evidencia.
 - **EvidenceModal.jsx (Visor de Evidencias multimedia):** Ventana emergente de alta prioridad que se renderiza condicionalmente en el DOM al hacer clic en "Analizar Evidencia". Coordina la reproducción de los archivos de audio procesados por la IA y despliega el grid con las tres capturas visuales capturadas por las cámaras del cuadrante para la toma rápida de decisiones.
 - **RulesModal.jsx (Consola de Configuración de IA):** Modulo de administración avanzada estructurado en pestañas interactivas. Gestiona toda la lógica de control del backend desde el frontend, permitiendo al usuario interactuar directamente con las cuatro capas del motor de clasificación: los conceptos del *Motor IA (NLP)*, las restricciones en *Etiquetas de Descarte*, la inyección de palabras clave críticas en *Guardarraíles* y el mapa de *Asignación de Unidades* de emergencia.
- **src/utills/ (Módulos de Soporte e Inyección Técnica)**
 - **Propósito Técnico:** Contiene código utilitario puro (funciones JavaScript/JSX aisladas que no renderizan componentes de forma directa).

- **Comportamiento:** Centraliza funciones reutilizables como los formateadores de fechas y horas para congelar el segundo exacto en que ingresa la alerta a la bitácora, constantes que definen los nombres fijos de los sectores/cuadrantes de Jilotepec y scripts para limpiar cadenas de texto antes de enviarlas al motor NLP.

Archivos Estructurales y de Estilo

- **src/index.css (Punto de Inyección de Diseño)**
 - **Propósito Técnico:** Archivo de estilos global de la aplicación. Es el encargado de inicializar las directivas fundamentales del framework utilitario, permitiendo que todo el dashboard interprete correctamente las clases oscuras, los esquemas flexbox y los grids dinámicos.
- **src/App.css (Estilos Específicos del Layout)**
 - **Propósito Técnico:** Hoja de estilos complementaria destinada a personalizaciones visuales muy puntuales que escapan a las clases nativas de Tailwind, como animaciones personalizadas para el parpadeo de las alertas críticas o personalizaciones del scroll de la bitácora.
- **src/App.jsx (Orquestador y Layout Maestro)**
 - **Propósito Técnico:** Es el componente raíz de toda la jerarquía de React. Se encarga de pintar la estructura general del dashboard (la barra superior con las métricas de analítica de alertas, el buscador táctico con los *Parámetros de Filtrado* y el área central del monitor). Controla los estados de apertura de los modales y actúa como el nodo que distribuye los flujos de información hacia las tarjetas de alertas.
- **src/main.jsx (Punto de Montaje del Sistema)**
 - **Propósito Técnico:** Es el disparador oficial del Frontend. Se encarga de importar la biblioteca de React, enlazar las hojas de estilo globales e instanciar el DOM virtual dentro del nodo contenedor HTML (<div id="root"></div>), envolviendo la aplicación en el modo estricto

(StrictMode) para auditar la calidad del renderizado durante el desarrollo.

Acceso a la Aplicación

El despliegue de esta aplicación está totalmente automatizado mediante contenedores Docker, lo cual garantiza que el entorno de ejecución sea idéntico tanto en desarrollo como en producción.

Una vez que el proyecto se ha inicializado correctamente utilizando el orquestador de contenedores (Docker Compose), la aplicación estará disponible inmediatamente a través del navegador web.

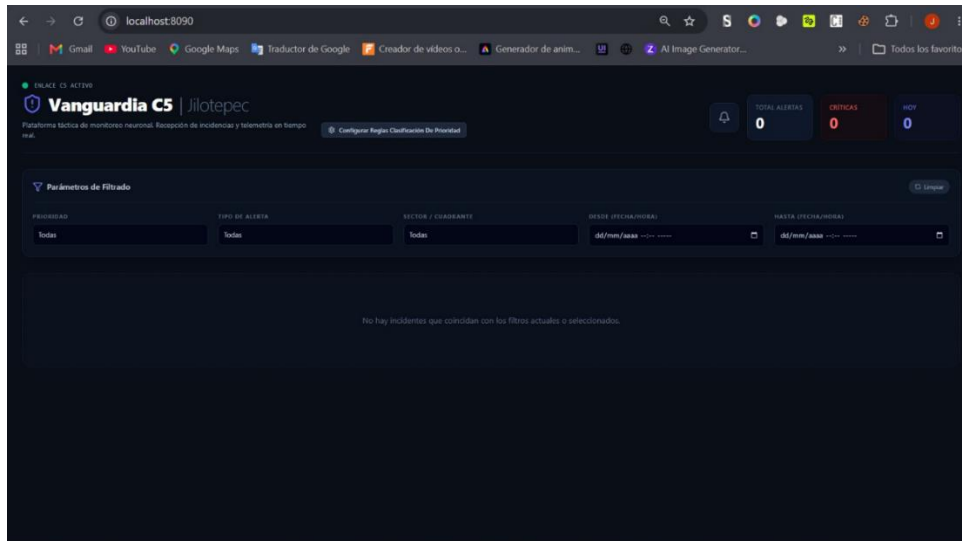
- **URL de acceso:** <http://localhost:8090>

Nota: Asegúrate de que el contenedor de Docker esté en estado Up antes de intentar acceder a la dirección mencionada. Si el puerto 8090 no responde, verifica el estado del servicio mediante el comando `docker ps`.

Capturas de pantalla de la interfaz

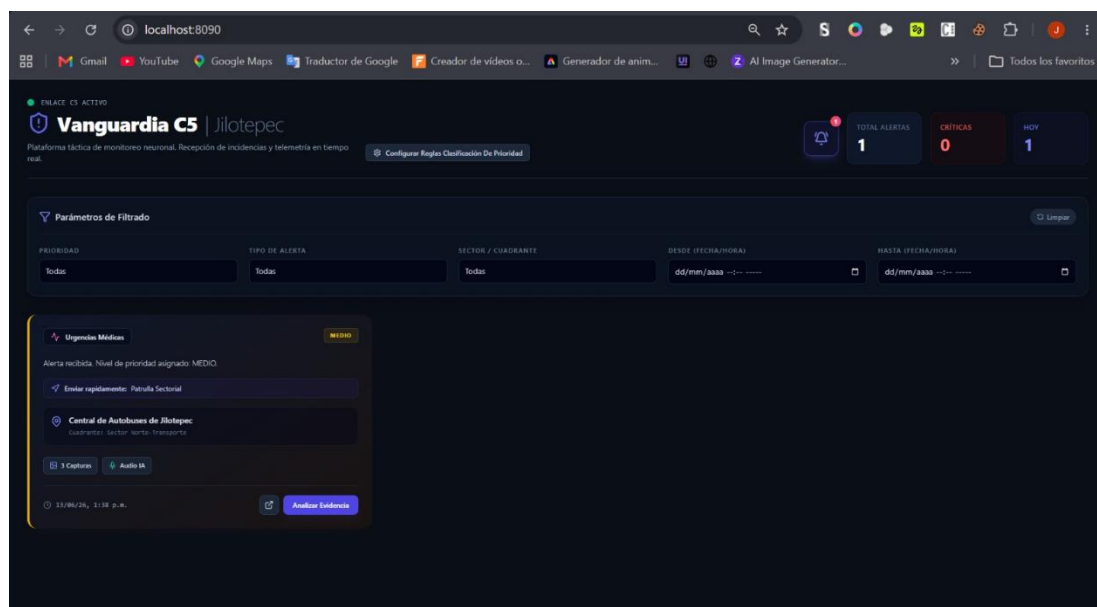
Recepción de Incidencias en Tiempo Real

- **Header:** Muestra el estado del sistema (ENLACE C5 ACTIVO) y el botón para configurar las reglas de la IA.
- **Métricas KPI:** Contadores rápidos para rastrear el volumen de reportes del turno (**Total Alertas, Críticas y Hoy**).
- **Filtros:** Selectores rápidos para segmentar incidencias por *Prioridad, Tipo, Sector y Fecha/Hora*.
- **Área Central:** Despliega dinámicamente el mensaje "*No hay incidentes...*" mientras espera la llegada de alertas en tiempo real.



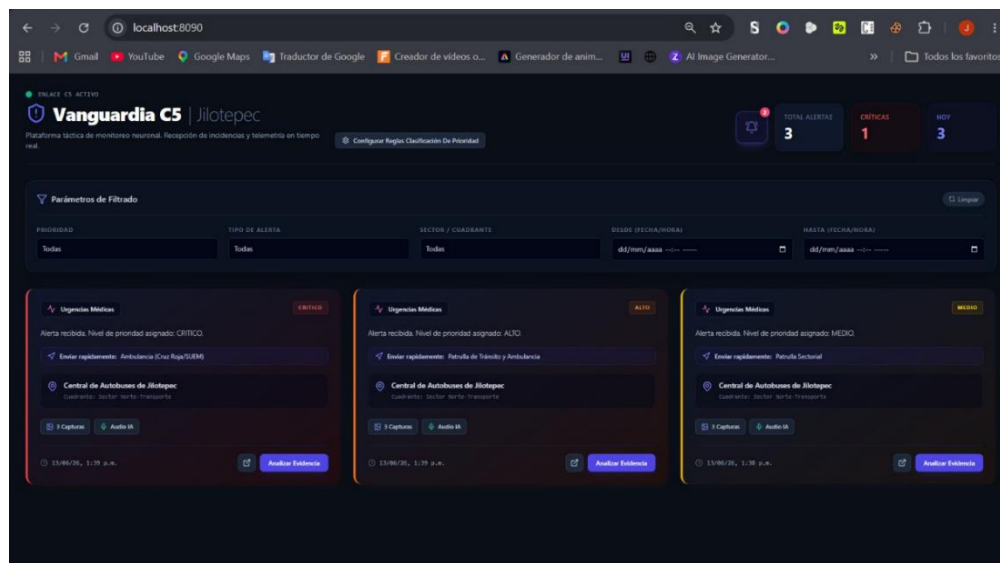
Grid Dinámico de Alertas y Niveles de Prioridad

- **Métricas Activas:** Los contadores reflejan la llegada del incidente incrementando a **1** el "Total Alertas" y "Hoy".
- **Tarjeta de Alerta (AlertCard.jsx):** Renderiza dinámicamente un reporte de *Urgencias Médicas* clasificado automáticamente con prioridad **Medio** (borde e indicador amarillos).
- **Datos del Incidente:** Muestra la ubicación (*Central de Autobuses de Jilotepec*), la sugerencia de despacho rápido (*Patrulla Sectorial*), los badges de archivos multimedia disponibles y la estampa de tiempo.
- **Acciones:** Incluye el botón "Analizar Evidencia" para desplegar los detalles y recursos de la alerta.



Grid de Alertas por Prioridad

- **Métricas:** Los contadores se actualizan a **3** en Total Alertas y **1** en Críticas.
- **Diseño en Rejilla:** Las tarjetas se ordenan en un flujo horizontal que optimiza el espacio del monitor.
- **Código de Colores (AlertCard.jsx):** Cada tarjeta adapta visualmente sus bordes y etiquetas según la severidad dictada por la IA:
 - **Rojo (CRÍTICO):** Requiere despacho inmediato de ambulancias de emergencia.
 - **Naranja (ALTO):** Requiere respuesta prioritaria de tránsito y apoyo médico.
 - **Amarillo (MEDIO):** Requiere atención estándar de la patrulla sectorial.

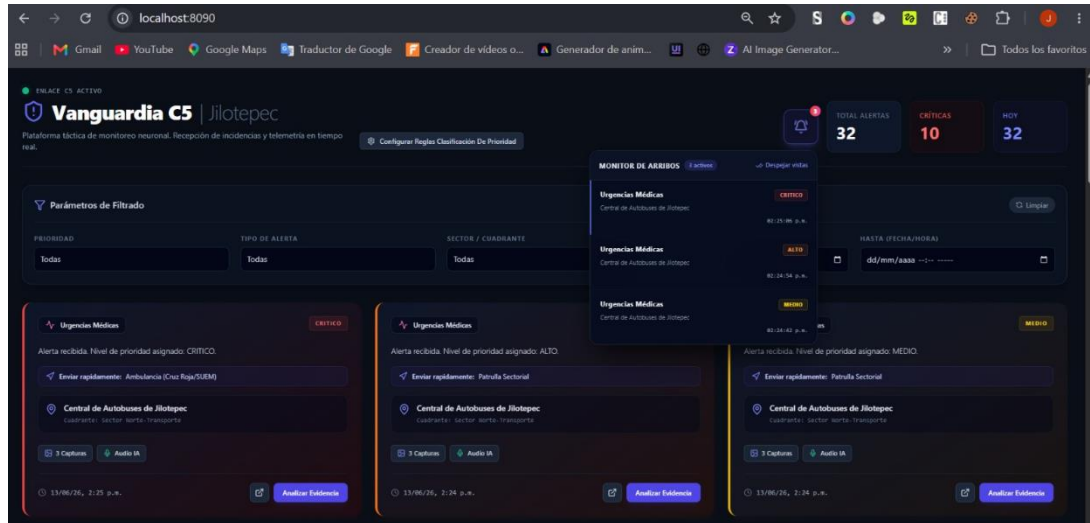


Monitor de Arribos Colectivo

- **Métricas Escaladas:** Los contadores reflejan un flujo mayor con **32** Total Alertas, **10** Críticas y **32** en el registro de Hoy.
- **Componente Monitor de Arribos:** Superpone una lista flotante compacta que resume las últimas incidencias entrantes en orden cronológico inverso.
- **Detalle por Alerta:** Cada fila del menú sintetiza el tipo de emergencia (*Urgencias Médicas*), la ubicación (*Central de Autobuses de Jilotepec*), la

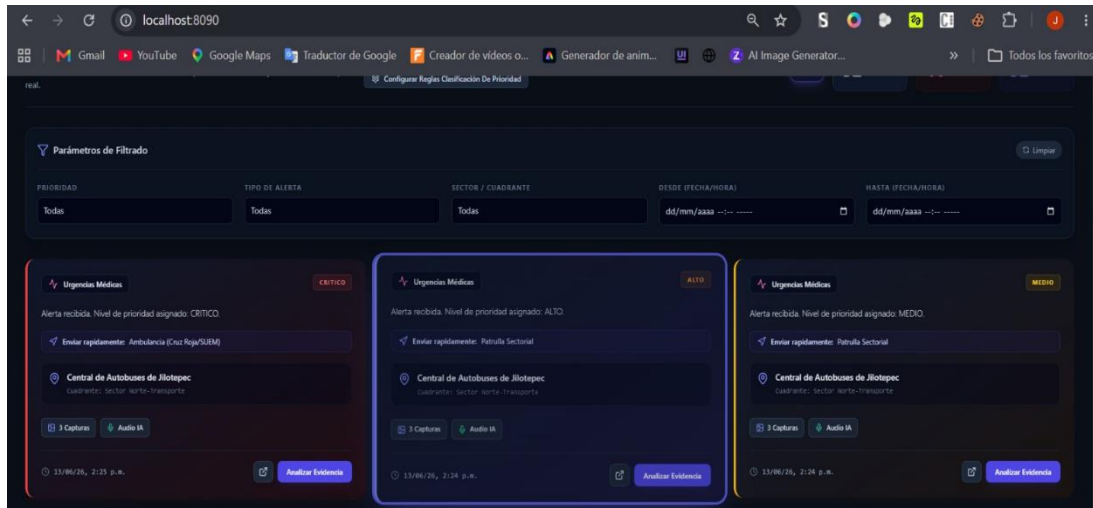
estampa de tiempo exacta con segundos y el distintivo de color de prioridad de la IA (*Crítico, Alto, Medio*).

- **Acción:** Incluye la opción rápida Despejar vistas en la esquina superior derecha para limpiar el historial de notificaciones visualizadas.



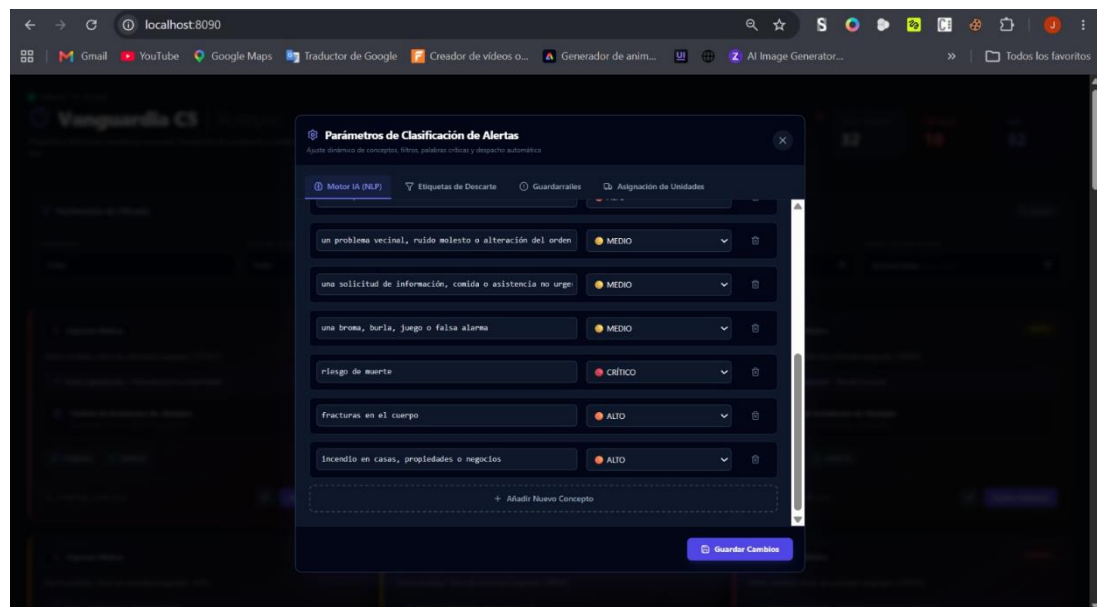
Foco de Selección en Tarjetas de Alerta

- **Efecto Hover / Focus Activo:** Al posicionar el cursor o seleccionar la tarjeta central (Prioridad Alto), el sistema activa dinámicamente un borde brillante con tonalidad azul/violeta.
- **Aislamiento de Navegación:** Este indicador visual ayuda al monitorista a mantener el foco en un incidente específico antes de accionar eventos críticos como la apertura de modales.
- **Persistencia de Estados:** Mientras una tarjeta mantiene el foco de atención, los componentes hermanos (AlertCard.jsx) de prioridad Crítica (Rojo) y Medio (Amarillo) atenúan sutilmente su presencia visual para evitar distracciones en el despacho.



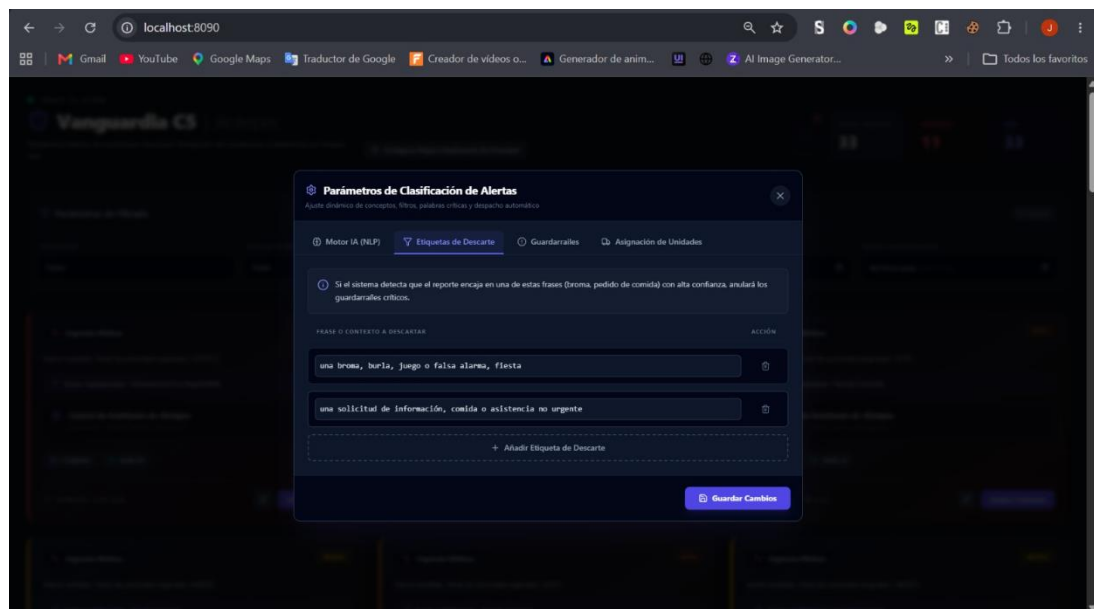
Consola de Reglas de IA (RulesModal.jsx)

- **Pestaña Motor IA (NLP):** Expone las frases y conceptos semánticos que procesa el algoritmo para clasificar incidencias.
- **Calibración:** Permite editar frases y asignarles su nivel de riesgo por dropdown (ej. "*riesgo de muerte*" **CRÍTICO**).
- **Estructura Modular:** Organiza en pestañas las cuatro capas del motor: *Motor IA*, *Etiquetas de Descarte*, *Guardarraíles* y *Asignación de Unidades*.
- **Acción:** El botón "Guardar Cambios" actualiza las directivas en el servicio correspondiente.



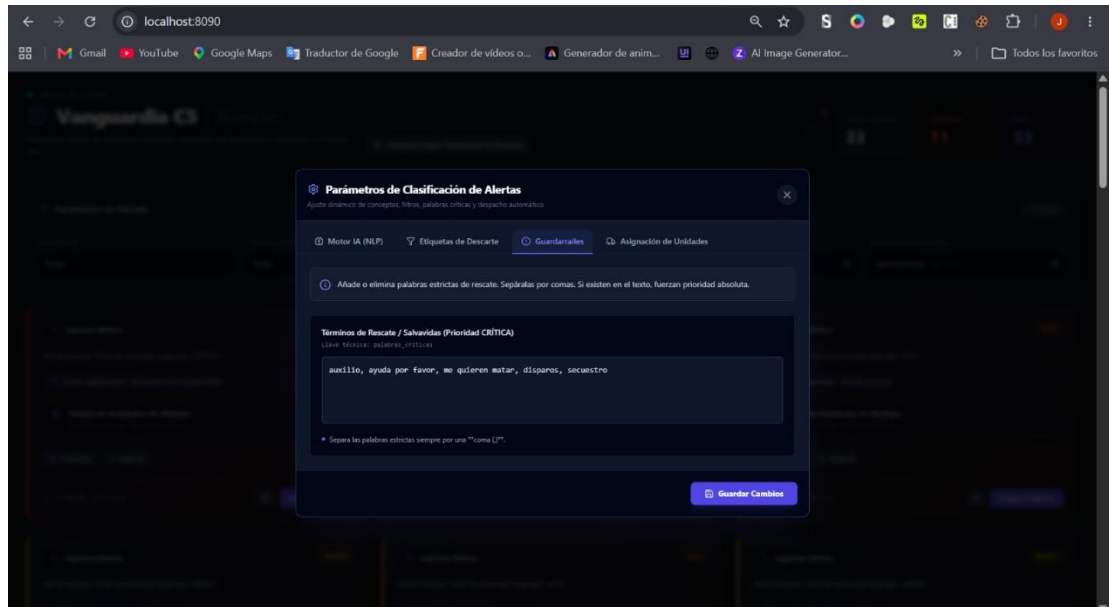
Filtros de Descarte (RulesModal.jsx)

- **Pestaña Activa:** *Etiquetas de Descarte*. Anula los guardarraíles críticos si el reporte encaja con alta confianza en contextos irrelevantes (ej. bromas o pedidos de comida).
- **Filtros:** Muestra las frases de depuración activas para omitir ruido operativo ("*una broma, burla...*" y "*una solicitud de información...*").
- **Acciones:** Permite la eliminación individual mediante el icono de basura o la expansión del diccionario con el botón + Añadir Etiqueta de Descarte.



Configuración de Guardarraíles (RulesModal.jsx)

- **Pestaña Activa:** *Guardarraíles*. Define términos estrictos de rescate que, de aparecer en el reporte, fuerzan de manera automatizada la prioridad máxima.
- **Términos de Rescate / Salvavidas:** Muestra la lista de palabras críticas activas mapeadas bajo la llave técnica palabras_criticas ("*auxilio, ayuda por favor, me quieren matar, disparos, secuestro*").
- **Regla de Sintaxis:** Requiere ingresar las palabras estrictas separadas por comas (,) dentro del área de texto para su correcta indexación en el motor de análisis.



Asignación de Unidades (RulesModal.jsx)

- **Pestaña Activa:** *Asignación de Unidades*. Vincula corporaciones operativas con palabras clave detectadas en la transcripción de la alerta.
- **Mapeo Corriente:**
 - **Bomberos y Protección Civil:** Filtra conceptos como *"incendio, fuego, quemando, humo, gas"*.
 - **Ambulancia (Cruz Roja/SUEM):** Filtra conceptos como *"médico, herido, sangre, desmayo, infarto, ambulanc"*.
 - **Patrulla de Tránsito y Ambulancia:** Filtra conceptos como *"choque, accidente de tránsito, atropellado"*.
- **Controles:** Permite eliminar mapeos individuales o estructurar nuevos flujos de despacho mediante el botón + Añadir Nueva Unidad de Respuesta.

